



UNIVERSIDAD AUTÓNOMA DE SAN LUIS POTOSÍ

FACULTAD DE ESTUDIOS
PROFESIONALES
ZONA MEDIA

INGENIERÍA MECATRÓNICA



PRÁCTICA 10

Lectura de Valores Analógicos con un Potenciómetro

MATERIA: MICROCONTROLADORES

DOCENTE: Ing. JESÚS PADRÓN

ALUMNO: CARLOS ANGEL GARCÍA SIFUENTES

CLAVE: A383917

CARRERA: INGENIERÍA MECATRÓNICA

SEMESTRE: 4° SEMESTRE

FECHA: 24 DE MAYO DE 2026

Lectura y Conversión de Señales Analógicas mediante el ADC

de 12 Bits de la Raspberry Pi Pico W con Potenciómetro

CARLOS ÁNGEL GARCÍA SIFUENTES

a383917@alumnos.uaslp.mx

Fecha: 20 de mayo de 2026

Resumen—Se presenta el desarrollo de la Práctica 10, cuyo objetivo es aprender a leer señales analógicas mediante el convertidor analógico-digital (ADC) de 12 bits integrado en la Raspberry Pi Pico W. Un potenciómetro de 10 kΩ conectado al pin GP26 (canal ADC0) genera una tensión variable entre 0 V y 3.3 V. El programa en lenguaje C lee el valor crudo del ADC (0–4095), lo convierte al voltaje equivalente mediante la fórmula de proporcionalidad del conversor, y transmite ambos valores al monitor serial de VS Code cada 500 ms. Los resultados verifican la resolución del ADC de aproximadamente 0.8 mV por nivel.

I. INTRODUCCIÓN

I-A Descripción General y Trabajo Previo

En todas las prácticas anteriores se trabajó exclusivamente con señales digitales: valores discretos de 0 y 1 lógico. En esta práctica se aborda por primera vez el dominio analógico, donde las magnitudes físicas como temperatura, presión o posición producen señales continuas que el microcontrolador no puede leer directamente. Para ello se estudió el funcionamiento de un convertidor analógico-digital (ADC) y el comportamiento de un potenciómetro.

Un **potenciómetro** es una resistencia variable con tres terminales: dos extremos fijos y un cursor móvil central. Al conectar los extremos a 3.3 V y GND, el cursor genera un voltaje proporcional a su posición angular, actuando como un divisor de voltaje ajustable que varía continuamente entre 0 V y 3.3 V.

I-B Señales Analógicas y Digitales

Una **señal analógica** es una función matemática continua que puede tomar infinitos valores dentro de un rango. Fenómenos físicos como la temperatura, la presión y la intensidad de luz son ejemplos de señales analógicas. Sus características principales son que se pueden procesar directamente en tiempo real, cuentan con dos

parámetros (amplitud y frecuencia) y son señales de alta densidad de información.

Una **señal digital** es de tipo discreto: solo toma valores en intervalos definidos, representados generalmente como ceros y unos en sistema binario. Sus parámetros son la altura de pulso (nivel eléctrico), la duración (ancho de pulso) y la frecuencia de repetición.

I-C Conversión Analógica a Digital (ADC)

El proceso de conversión analógico-digital comprende tres etapas fundamentales:

- **Muestreo:** se toma la señal analógica y se mide en intervalos de tiempo regulares. La frecuencia de muestreo debe cumplir el Teorema de Nyquist: debe ser al menos el doble de la frecuencia máxima presente en la señal para evitar el fenómeno de *aliasing*.
- **Cuantización:** cada muestra se asigna al valor discreto más cercano dentro de un rango determinado. La precisión depende de la resolución del conversor: un ADC de 8 bits tiene 256 niveles, mientras que uno de 12 bits tiene 4096.
- **Codificación:** los valores cuantizados se representan en código binario, obteniendo la representación digital de la señal.

I-D El ADC de 12 Bits de la Raspberry Pi Pico W

La Raspberry Pi Pico W integra un ADC de 12 bits accesible en los pines GP26, GP27 y GP28, con una tensión de referencia de 3.3 V. Esto implica un rango de salida de 0 a 4095 niveles. La relación entre el voltaje de entrada y el valor digital es:

$$V_{pot} = \frac{\text{Valor ADC} \times 3.3}{4095} \quad (1)$$

$$\text{Valor ADC} = \frac{V_{pot} \times 4095}{3.3} \quad (2)$$

de voltaje detectable, es:

$$\text{Resolución} = \frac{3.3\text{V}}{4095} \approx 0.806\text{ mV/nivel} \quad (3)$$

II. DESARROLLO

II-A Material y Recursos Utilizados

- **Hardware:** 1 Placa Raspberry Pi Pico W, 1 cable micro USB tipo B, 1 potenciómetro de 10 kΩ, 1 protoboard y alambre calibre 22 AWG.
- **Software y librerías:** SDK de Raspberry Pi Pico, Pico – Visual Studio Code, extensión Serial Monitor, librerías `pico/stdlib.h` y `hardware/adc.h`.

II-B Esquemático del Circuito

El potenciómetro se conecta con su terminal izquierdo al pin 37 (3V3OUT, 3.3V) de la Pico W, su terminal derecho al pin 33 (GND), y su cursor central al pin 31 (GP26, canal ADC0). La alimentación de la placa se realiza por el puerto USB para poder transmitir datos al monitor serial de la computadora.

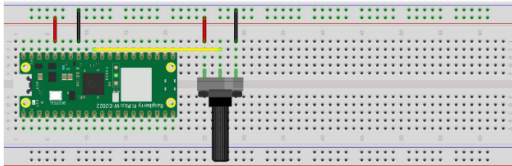


Figura 1: Esquemático del circuito: potenciómetro de 10 kΩ conectado al ADC0 (GP26) con alimentación desde los pines 3V3OUT y GND de la Pico W.

II-C Estructura del Proyecto y Configuración de CMake

Una característica nueva en esta práctica es la presencia de dos líneas adicionales en el `CMakeLists.txt` que habilitan explícitamente la salida serial por USB y deshabilitan la UART, lo que permite visualizar los datos en el monitor serial de VS Code sin necesidad de hardware adicional.

ARCHIVO CMakeLists.txt

```
cmake_minimum_required(VERSION 3.13)

include(pico_sdk_import.cmake)
set(PICO_BOARD pico_w)

project(PRACTICA_10)
```

```
pico_sdk_init()

add_executable(ADC
    ADC.c
)

target_link_libraries(ADC
    pico_stdlib
    hardware_adc
)

pico_enable_stdio_usb(ADC 1)    % Habilita
    serial por USB
pico_enable_stdio_uart(ADC 0)   %
    Deshabilita serial por UART

pico_add_extra_outputs(ADC)
```

La librería `hardware_adc` es nueva respecto a prácticas anteriores y provee las funciones `adc_init()`, `adc_gpio_init()` y `adc_read()` necesarias para operar el conversor.

II-D Implementación del Código en C

CÓDIGO FUENTE – ADC.c

```
#include <stdio.h>
#include "pico/stdlib.h"
#include "hardware/adc.h"

#define POT_PIN 26
#define ADC_MAX_VALUE 4095
#define ADC_VREF 3.3f

int main() {
    stdio_init_all();
    adc_init();
    adc_gpio_init(POT_PIN);
    adc_select_input(0);

    while (1) {
        uint16_t raw_value = adc_read();
        float voltage = (raw_value *
            ADC_VREF) / ADC_MAX_VALUE;

        printf("Valor_ADC:_%d,_Voltaje:_%0.2f_V\n",
            raw_value, voltage);

        sleep_ms(500);
    }
}
```

II-D1. Funcionamiento del código

El programa sigue la siguiente secuencia de operaciones:

1. `stdio_init_all()`: inicializa la comunicación USB-serial, habilitando `printf()` para enviar datos a la computadora.
2. `adc_init()`: enciende y configura el hardware del ADC interno del RP2040.
3. `adc_gpio_init(26)`: configura el pin GP26 en modo analógico, deshabilitando sus buffers digitales para evitar que interfieran con la lectura del ADC.
4. `adc_select_input(0)`: selecciona el canal ADC0, que corresponde físicamente al pin GP26.
5. En el lazo `while(1)`, `adc_read()` captura una muestra del ADC devolviendo un entero de 12 bits entre 0 y 4095.
6. La conversión a voltaje se realiza aplicando la ecuación (1): se multiplica el valor crudo por 3.3 V y se divide entre 4095.
7. `printf()` transmite el valor crudo y el voltaje calculado al monitor serial cada 500 ms.

II-E Configuración del Monitor Serial en VS Code

Para visualizar los datos transmitidos por la Pico W es necesario configurar el monitor serial integrado en Pico – Visual Studio Code:

1. Verificar que la extensión **Serial Monitor** esté instalada en VS Code.
2. Conectar la Pico W por USB con el programa ya cargado y el circuito armado.
3. Abrir la pestaña **Serial Monitor** en el panel de terminal de VS Code.
4. Seleccionar el puerto COM asignado al microcontrolador (generalmente COM5; para identificarlo, desconectar la placa y verificar qué puerto desaparece de la lista).
5. Configurar el *Baud rate* en **115200**.
6. Presionar *Start Monitoring*.

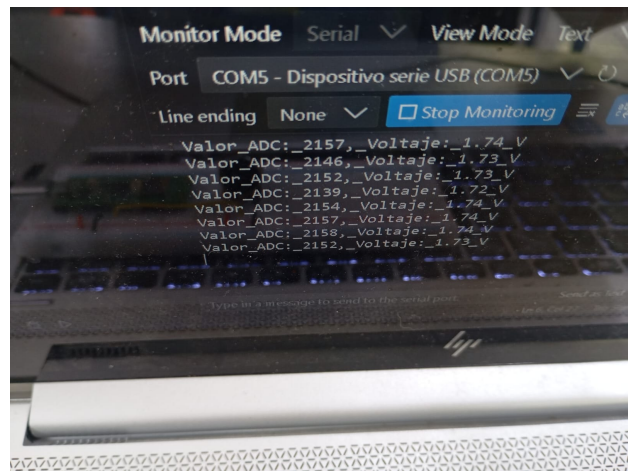


Figura 2: Monitor serial en VS Code mostrando el valor ADC y su equivalente en voltaje actualizados cada 500 ms.

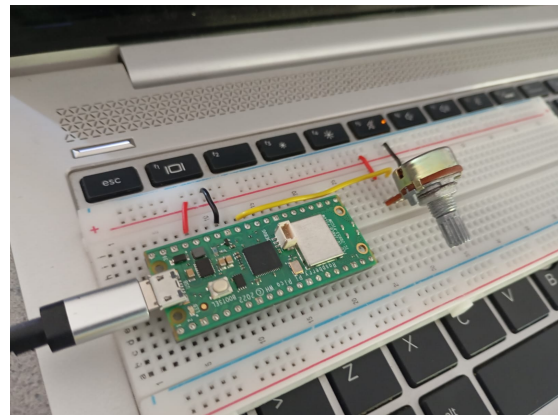


Figura 3: Circuito físico en funcionamiento con el potenciómetro conectado al pin GP26 de la Raspberry Pi Pico W.

III. PREGUNTAS DE REPASO

III-A Si el voltaje del potenciómetro es 1V, ¿qué valor de ADC representará?

Aplicando la ecuación (2):

$$\text{Valor ADC} = \frac{1\text{ V} \times 4095}{3.3\text{ V}} = \frac{4095}{3.3} \approx 1241 \quad (4)$$

Un voltaje de 1V produce un valor digital de **1241** en el ADC de 12 bits.

III-B ¿Cuál es la representación binaria de un voltaje de 2 V?

Primero se obtiene el valor ADC para 2V:

$$\text{Valor ADC} = \frac{2V \times 4095}{3.3V} = \frac{8190}{3.3} \approx 2482 \quad (5)$$

Convirtiendo 2482 a binario:

$$2482_{10} = 1001\ 1011\ 0010_2 \quad (6)$$

Esta es la palabra de 12 bits que el ADC enviaría al microcontrolador para una entrada de 2 V.

III-C Si el ADC de 12 bits tiene 4096 niveles, ¿por qué se usa 4095 en la ecuación?

Un ADC de 12 bits puede representar $2^{12} = 4096$ valores distintos, numerados del 0 al 4095. El valor máximo que puede producir el conversor es **4095** (no 4096), que corresponde al voltaje de referencia máximo (3.3 V). Si se usara 4096 en el denominador, la fórmula daría un voltaje calculado menor al real cuando el ADC lee 4095, introduciendo un error sistemático. Usar 4095 en el denominador garantiza que cuando `adc_read()` devuelve 4095, el voltaje calculado sea exactamente 3.3 V, que es el valor físico correcto.

III-D ¿Cuál es la resolución de un ADC de 16 bits?

Un ADC de 16 bits con la misma referencia de 3.3 V tiene $2^{16} = 65536$ niveles (0 a 65535). Su resolución es:

$$\text{Resolución}_{16\text{ bits}} = \frac{3.3V}{65535} \approx 50.4\ \mu\text{V/nivel} \quad (7)$$

Comparado con los 0.806 mV/nivel del ADC de 12 bits de la Pico W, el ADC de 16 bits es aproximadamente **16 veces más preciso**, permitiendo detectar variaciones de voltaje de apenas 50 μ V.

III-E Conversión de valores binarios a voltaje

Para cada valor binario se aplica la ecuación (1): $V = (\text{Valor ADC} \times 3.3)/4095$.

Tabla I: Conversión de valores binarios del ADC a voltaje equivalente

Binario (12 bits)	Decimal	Cálculo	Voltaje
1101 1101 0101	3541	$\frac{3541 \times 3.3}{4095}$	2.855 V
0010 0110 0101	613	$\frac{613 \times 3.3}{4095}$	0.494 V
0000 1010 1111	175	$\frac{175 \times 3.3}{4095}$	0.141 V
1101 0110 1101	3437	$\frac{3437 \times 3.3}{4095}$	2.771 V

III-F Explicación del funcionamiento del código C

El programa comienza inicializando la comunicación serial USB con `stdio_init_all()`, lo que permite el envío de datos a la computadora mediante `printf()`. A continuación, `adc_init()` enciende el hardware del ADC interno, `adc_gpio_init(26)` configura el pin GP26 como entrada analógica pura (deshabilitando sus buffers digitales), y `adc_select_input(0)` conecta el multiplexor interno del ADC al canal 0, que corresponde físicamente a GP26.

Dentro del lazo `while(1)`, la función `adc_read()` captura una muestra instantánea de la señal analógica en el pin GP26 y devuelve un entero de 16 bits sin signo (`uint16_t`) con los 12 bits inferiores válidos, representando el valor digitalizado entre 0 y 4095. Este valor se convierte a voltaje multiplicándolo por 3.3 V y dividiéndolo entre 4095, obteniendo un número de punto flotante con la tensión real medida. Finalmente, `printf()` transmite ambos valores por el puerto USB-serial y `sleep_ms(500)` genera una pausa de 500 ms antes de la siguiente lectura, produciendo aproximadamente dos muestras por segundo visibles en el monitor serial.

IV. CONCLUSIONES

- El ADC de 12 bits de la Raspberry Pi Pico W permite leer señales analógicas con una resolución de aproximadamente 0.806 mV por nivel, lo cual es suficiente para la mayoría de aplicaciones de sensado en ingeniería mecatrónica, como lectura de temperatura, posición angular o intensidad de luz.
- La librería `hardware_adc` del SDK simplifica significativamente la configuración del conversor, encapsulando en pocas funciones (`adc_init()`, `adc_gpio_init()`, `adc_select_input()`, `adc_read()`) una secuencia de operaciones que

de otro modo requeriría manipular directamente los registros del hardware.

- Las líneas `pico_enable_stdio_usb()` y `pico_enable_stdio_uart()` en el `CMakeLists.txt` resultaron fundamentales para habilitar la comunicación serial por USB, demostrando que la configuración del sistema de compilación tiene impacto directo en las capacidades de comunicación del programa.
- La fórmula de conversión $V = (ADC \times 3.3)/4095$ establece una relación lineal directa entre el valor digital y el voltaje analógico, lo que facilita escalar la misma lógica a otros sensores con diferentes rangos simplemente ajustando el factor de escala.
- Como mejora futura se podría implementar un promediado de múltiples lecturas consecutivas para reducir el ruido en la señal ADC, o mapear el valor leído a una escala física significativa (como grados Celsius para un sensor de temperatura) en lugar de mostrar únicamente el voltaje bruto.

ANEXO B – CMakeLists.txt (completo)

```
cmake_minimum_required(VERSION 3.13)

include(pico_sdk_import.cmake)
set(PICO_BOARD pico_w)

project(PRACTICA_10)

pico_sdk_init()

add_executable(ADC
    ADC.c
)

target_link_libraries(ADC
    pico_stdlib
    hardware_adc
)

pico_enable_stdio_usb(ADC 1)
pico_enable_stdio_uart(ADC 0)

pico_add_extra_outputs(ADC)
```

ANEXOS

A. Código fuente – ADC.c

ANEXO A – ADC.c (completo)

```
#include <stdio.h>
#include "pico/stdlib.h"
#include "hardware/adc.h"

#define POT_PIN 26
#define ADC_MAX_VALUE 4095
#define ADC_VREF 3.3f

int main() {
    stdio_init_all();
    adc_init();
    adc_gpio_init(POT_PIN);
    adc_select_input(0);

    while (1) {
        uint16_t raw_value = adc_read();
        float voltage = (raw_value *
            ADC_VREF) / ADC_MAX_VALUE;
        printf("Valor_ADC:_%d,_Voltaje:_.%2
            f_V\n",
            raw_value, voltage);
        sleep_ms(500);
    }
}
```

B. Archivo CMakeLists.txt

C. Diagrama en Protoboard

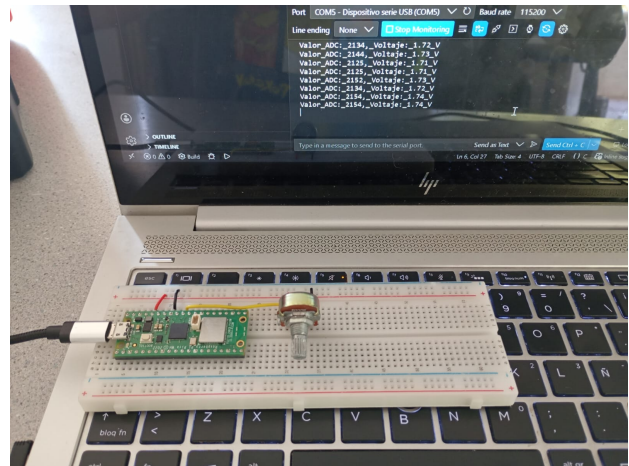


Figura 4: Diagrama en protoboard: potenciómetro de 10 kΩ conectado a GP26 con alimentación desde los pines 3V3OUT (pin 37) y GND (pin 33) de la Pico W.